

Étapes de l'algorithme d'affectation

Ce document décrit, étape par étape, l'algorithme d'affectation utilisé dans l'application, depuis les entrées jusqu'au résultat final.

Contexte - But : assigner des éléments (tâches, ressources, personnes) à des cibles selon des critères (coût, capacité, priorité). - Entrées : liste d'éléments à affecter, liste de cibles, matrice de coût/score, contraintes éventuelles.

Résumé des étapes (vue d'ensemble) 1. Lecture des données d'entrée (éléments, cibles, contraintes). 2. Initialisation des structures (tableaux de coûts, variables d'état, résultats partiels). 3. Boucle principale : recherche d'une meilleure affectation - Pour chaque élément non affecté, évaluer les scores pour chaque cible admissible. - Choisir la meilleure cible selon le critère (minimiser coût ou maximiser score). - Vérifier les contraintes (capacités, exclusivités) et appliquer la décision. - Mettre à jour les structures et marquer l'élément comme affecté. 4. Améliorations/itérations : appliquer un raffinement (ex. échange pair-à-pair, algorithme hongrois, heuristique locale) pour améliorer la solution. 5. Critère d'arrêt : tous les éléments affectés ou plus d'améliorations possibles / itérations atteintes. 6. Calcul des métriques finales (coût total, score total, violation de contraintes) et préparation du résultat.

Détail étape par étape

1) Lecture des entrées - Charger les listes depuis le modèle ou le backend. - Valider la cohérence (pas d'éléments ou cibles manquantes, coûts définis).

2) Initialisation - Créer une table `affectation` vide pour stocker les paires (élément → cible). - Initialiser `capacite\_restante` pour chaque cible si applicable. - Préparer une file ou liste d'éléments à traiter.

3) Première passe d'affectation (heuristique gloutonne) - Pour élément e dans la liste non affectée: 1. Calculer `score(e, c)` ou `cost(e, c)` pour chaque cible c admissible. 2. Filtrer les cibles qui ne respectent pas les contraintes (ex. capacité atteinte). 3. Sélectionner $c_best = \argmin_cost$ ou $\argmax_score$. 4. Si c_best existe: affecter $e \rightarrow c_best$, décrémenter $capacite_restante[c_best]$. 5. Sinon: marquer e non affecté (ou envoyer en file d'attente de résolution manuelle).

4) Raffinement (optionnel mais recommandé) - Application d'une stratégie d'amélioration: - Algorithme hongrois (pour problèmes d'affectation carrés et coût) pour obtenir solution optimale. - Recherche locale (swap) : parcourir paires d'affectations et tester échanges qui réduisent le coût. - Recuit simulé / métaheuristiques pour grands problèmes.

5) Vérification et clôture - Vérifier que toutes les contraintes sont respectées. - Calculer métriques finales (coût total, nombre d'éléments non affectés). - Préparer le format de sortie (JSON, affichage UI, export).

Exemple de sortie - `affectation = [{"elementId": "E1", "cibleId": "C3"}, ...]` - `metrics = {"totalCost": 123.4, "unassigned": 0}`

Références dans le projet - Composant principal : src/app/affectation/affectation.component.ts

--- Fichier généré automatiquement pour export PDF / PowerPoint.